Propagation Logic:
A Single-Operator Foundation for Logic and Calculus

James Pugmire
Independent Researcher
Version 13

Abstract

Propagation Logic (PL) is a formal system built on a single dynamical mechanism: as relational patternings propagate they accumulate a loaded history relative to the gradient fields available in their context, and this loaded history sets the gradient demands for all further propagation. Patterns with less loaded history propagate faster. As complexity builds it generates relational drag. When demands exceed available gradients, patterns reconfigure toward coherence. At maximum complexity — when a pattern's history includes itself — reconfiguration pressure becomes unbounded, and simpler structures propagate outward from the complex core.

Propositional, first-order, modal, and probabilistic logic — and the core operations of calculus — are each the boundary conditions forced by propagation over a specific carrier with a specific gradient family. Each system formalises what propagation looks like when relational structure is parsed through a particular subject-predicate cut and the output is treated as a set of discrete entities with intrinsic properties. PL operates at the level of propagation that precedes that cut.

A distinction that formal treatments of PL frequently obscure is made explicit here: gradient fields G are not a different kind of thing from patterns P. G is a pattern occupying the gradient role contextually — a local and momentary assignment, not an intrinsic difference of substance. All claims to describe a gradient field are themselves propagation events in a meta-context. This G/P distinction closes the ontological loop of the mechanism and prevents gradient fields from being smuggled in as hidden substances.

A Python implementation verifies all structural claims to machine precision.

Keywords: propagation logic, foundations of mathematics, automatic differentiation, modal logic, Gödel incompleteness, probabilistic logic, fundamental theorem of calculus, G/P distinction, process ontology

MSC 2020: 03B05, 03B45, 03F40, 03A05, 26A06, 65D25, 68T27

Conflict of interest: None. Funding: None.

Note on the Subject-Predicate Cut and Reification

Every formal system currently in use begins after a cut has already been made. The cut is the subject-predicate decision: the parsing that takes relational structure and produces discrete objects with identity and intrinsic properties. Once that cut is made, A = A appears self-evident, the binary arithmetic of {0,1} appears natural, and the foundational laws appear to be bedrock. They are not bedrock. They are boundary conditions that become visible only once you have decided to parse here rather than there.

That decision is ad hoc in a precise sense: it is imported without derivation, necessary for the formal system to start, and impossible to justify using the system's own tools. The identity law A = A is not a primitive — it is a relational boundary condition, constituted by the equivalence relation, the symbols, and the interpretive context required to state it. The law of non-contradiction and the law of excluded middle are arithmetic facts about the two-element carrier that the subject-predicate cut produces — not truths about the world that axioms capture.

When the output of the cut is treated as a self-standing entity with intrinsic properties — when the parsed structure is reified — the relational constitution of that entity is forgotten. Classical logic, set theory, and standard mathematics all begin with reified outputs and build upward. This is productive. It is also why these systems cannot account for their own foundations from inside themselves.

PL operates before the cut. Its primitive is propagation: relational patternings moving through contexts, accumulating history, generating gradient demands, reconfiguring toward coherence. Stability is what propagation produces under specific carrier and gradient conditions. It is not assumed.

The same reification that applies to patterns applies equally to gradients. A gradient field G — the rule governing how a pattern transforms — is not a substance separate from the propagation process. G is a pattern occupying the gradient role contextually: a relational configuration that has been constituted by prior propagation, assigned the G role in a specific interaction, and capable of occupying the P role in the next. The distinction between pattern and gradient is a local cut, not an ontological divide. Section 2.6 makes this explicit. Any description of the mechanism is itself an instance of the mechanism — there is no view from outside, only propagation describing propagation in a meta-context.

Any evaluation of PL that imports classical standards — discrete objects, intrinsic identity, truth as primitive — is applying the output of the cut to the process that preceded the cut. The result is not falsification. It is a demonstration that those standards are themselves boundary conditions of a particular parsing.

---

1. Introduction

As relational patternings propagate they gain a loaded history relative to the gradient fields available in their context, and they gain complexity with this loaded history. The loaded history sets the gradient demands for all further propagation, relative to the available gradients of the context. This is the core mechanism. Everything in this paper is a consequence of it.

Terminology note: 'relational patterning' is the mechanism-layer description of what the model layer calls a 'pattern' — a pair (v_P, L_P). The narrative uses 'relational patterning' to foreground that patterns are not substances with intrinsic properties but relational configurations constituted

through propagation. Formally, the primitive is always the pair (v_P, L_P) as defined in Definition 2.1.

The mechanism is called differential propagation because propagation is not uniform. Among patterns whose load exceeds the coherence threshold, those with less loaded history propagate toward coherence faster than more complex structures (Theorem 2.1). As complexity builds through propagation it generates relational drag. When demands exceed available gradients, patterns reconfigure toward more coherent structuring. At maximum complexity — when a pattern's history includes itself — reconfiguration pressure becomes unbounded, and simpler structures propagate outward.

One point requires emphasis that is often elided: gradient fields G are not a different kind of thing from patterns P. G is a pattern occupying the gradient role in a specific interaction — a contextual assignment, not an intrinsic substance difference. This is stated precisely in Section 2.6 and is the key addition in this version of the paper.

The mechanism in one statement

Relational patternings propagate differentially through contexts, accumulating loaded history that sets their gradient demands, generating relational drag with other structures, reconfiguring toward coherence when demands exceed available gradients, and producing outward propagation of simpler structure when complexity becomes unstable. Logic, calculus, and the dynamics of complex systems are all special cases. The mechanism contains no primitive substances — gradient fields included.

The formal operator through which this mechanism is expressed is:

$$\mathbf{P} \mathbin{/_C} \mathbf{G} := \mathbf{Q}$$

A loaded pattern P is constituted by a value component $v_P \in V$ and a loaded history H_P whose magnitude $L_P = |H_P|$ is the informational load. A gradient field $G \in \Gamma_C$ specifies how both components transform as P propagates through context C. The output Q is the updated relational configuration. v_P is the designation relation; L_P is the propagation cost. These roles must not be conflated.

Logic and calculus differ only in carrier set and gradient family. The boundary conditions forced by differential propagation over a two-element carrier are not axioms imposed from outside — they are the exhaustive description of what can cohere in that space. The Unification Theorem (Section 13.5) states this precisely.

2. The Mechanism of Differential Propagation

This section defines the core structures of differential propagation. Everything in later sections is a special case of what is defined here. The definitions are ordered by the mechanism itself: pattern, history, demand, rate, drag, the propagation operator, the G/P distinction, reconfiguration, complexity growth, fixed points, and maximum complexity.

2.1 Loaded Patterns

Definition 2.1 — Loaded Pattern

A loaded pattern is a pair $P = (v_P, H_P)$ with $v_P \in V$, H_P a propagation history.

V is the carrier set. When propagation is parsed through the two-element subject-predicate cut, V = {0,1}. For propagation over a continuous real carrier, V = ℝ.

v_P is the designation component: the current relational state of propagation ($v_P = 1$: designated; $v_P = 0$: undesignated).

H_P is the loaded history: the accumulated record of every gradient field the pattern has propagated through. The informational load is the scalar projection: $L_P = |H_P| \geq 0$.

H_P is the conceptual foundation — the full propagation record. L_P is the operational measure — its magnitude. All computations operate on L_P. $L_P = 0$: seed state (no propagation history). $L_P > 0$: the pattern has accumulated history. It is in motion.

A pattern's gradient demands are its history speaking.

2.2 Gradient Demand

Definition 2.2 — Context, Support, and Gradient Demand

A context is a triple $C = (\Gamma_C, o_C, \theta_C)$ where: Γ_C is the set of gradient fields available in C; o_C is the load-combination rule; θ_C is the coherence threshold.

support(P, C) := $\min(L_P, \theta_C)$ — the portion of L_P that C can absorb.

demand(P, C) := $L_P - \text{support}(P, C) = \max(0, L_P - \theta_C)$. demand = 0: coherent. demand > 0: reconfiguration pressure.

$\text{designated}_C(P) := v_P = 1$ $\quad$ $\text{coherent}_C(P) := L_P \leq \theta_C$ $\quad$ $\text{valid}_C(P)$:= designated AND coherent

The two-element full-gradient-family setting (V = {0,1}, $\theta_C = 1$) produces the boundary conditions that classical logic formalises. V, θ_C, and Γ_C are the three parameters chosen to specify a logic. They are inputs, not derived outputs of the mechanism.

2.3 Propagation Rate

Definition 2.3 — Propagation Rate

rate(P, C) := support(P, C) / L_P for $L_P > 0$; = 1 for $L_P = 0$ (seed state).

rate = 1 when $L_P \leq \theta_C$ (fully supported, coherent). Rate < 1 when $L_P > \theta_C$. Lower load → higher rate.

Theorem 2.1 — Propagation Ordering

Among incoherent patterns P, Q in context C ($L_P, L_Q > \theta_C$):

rate(P, C) > rate(Q, C) $\iff L_P < L_Q$

Proof: for $L > \theta_C$: rate = θ_C/L. $\theta_C/L_P > \theta_C/L_Q \iff L_P < L_Q$ (since $\theta_C > 0$). □

Consequence: when multiple incoherent patterns reconfigure simultaneously, simpler patterns (lower load) reach coherence first. The propagation ordering is determined entirely by loaded history.

2.4 Relational Drag and the Zero-Drag Regime

Definition 2.4 — Relational Drag

When patterns P and Q propagate together through context C:
total load = $L_P + L_Q$ + drag(P, Q, C)
drag(P, Q, C) ≥ 0 is the load generated by the interaction of H_P and H_Q under Γ_C.

Zero-drag regime: drag(P, Q, C) = 0 for all P, Q, C. Total load = $L_P + L_Q$ exactly. This paper operates in the zero-drag regime throughout unless stated otherwise.

Nonzero-drag regimes characterise resource-sensitive logics: linear logic (each formula used exactly once — drag = ∞ beyond first use); relevance logic (premises must share gradient history). These are nonzero-drag settings of the same mechanism.

2.5 The Propagation Operator

Definition 2.5 — Propagation Operator

Given context C, loaded pattern P, and gradient field $G \in \Gamma_C$:
$P /_C G := Q = (v_Q, H_Q)$
where: v_Q is determined by applying G to v_P; $H_Q = H_P \oplus \text{record}(G)$ (history extended by G's propagation record); $L_Q = |H_Q|$ (load is the magnitude of the extended history).

Operationally: each gradient field G specifies its load-combination rule, which is the effect of ⊕ record(G) on L_P. G_{neg}: $L_Q = L_P$ (zero-load record). G_{and}: $L_Q = L_P + L_Q$ (additive record). G_{pow}: $L_Q = L_P \cdot n \cdot v^{(n-1)}$ (scaled record).

Every propagation step extends the history. Every history sets the demands for the next step. The operator is total for all P and all $G \in \Gamma_C$.

The propagation operator is the single primitive. It does not assume logical structure. Logical structure is what emerges when specific gradient families are chosen and the accumulated histories of propagating patterns generate specific boundary conditions.

2.6 The G/P Distinction: Gradient as Contextually Assigned Pattern

The key ontological clarification of this paper

The notation P/G→Q presents G as occupying a different ontological role from P. This appearance requires explicit correction. G is not a different kind of thing from P.

All G is P post-boundary-imposition. A gradient field G is a pattern occupying the gradient role in a specific interaction. G has loaded history. G was constituted by prior propagation. G can itself become the P in a higher-order propagation event. When we write G_{neg} or G_{and}, we are treating these as objects of study — patterns in the space of gradient families, propagating through a meta-level context. The gradient role is contextually assigned, not intrinsically held.

The P/G distinction is local and momentary. When two patterns meet in context C, one contextually occupies the G role (specifying the transformation) and the other the P role (undergoing it). After the interaction, the roles can flip: what was G becomes P for the next

propagation event, what was P can become the gradient for another. The distinction is a relational cut at the moment of interaction, not a permanent assignment of substance.

All claims to Non-G are meta-G. Any statement describing a gradient field — including this sentence — is itself a propagation event. 'G_{neg} flips the value component' is a pattern propagating through a meta-level gradient that specifies what gradient fields do. The description of G is itself G, operating one level up. There is no view from outside the mechanism.

Meta-claims collapse back. 'The mechanism generates classical logic' is not a statement made from outside the mechanism — it is $P/G \rightarrow Q$ running in a meta-context whose carrier is the space of formal systems. The loaded history of that statement includes the mechanism's output. This is not circularity. It is the self-consistency of the mechanism: any sufficiently expressive description of propagation is itself an instance of propagation.

Consequence for ontology. The mechanism contains no primitive substances. Not even gradient fields. The only primitive is the propagation event itself — the local relational event $P/G \rightarrow Q$ in which P undergoes transformation guided by G. Both P and G are, at the level of the meta-context, patterns. The subject-predicate cut that formal systems apply to P's output can equally be applied to G's constitution. Both are reifications of aspects of the same propagation process.

A practical consequence: when Sections 3–9 define gradient fields G_{neg}, G_{and}, G_{or}, and so on, these definitions are patterns in the meta-carrier propagating through the meta-gradient of formal specification. They occupy the G role in logical propagation events and the P role in the meta-propagation events of this paper. The dual role is not a paradox — it is the mechanism operating at two levels simultaneously, exactly as it does whenever complexity builds through iteration.

2.7 Reconfiguration

Definition 2.6 — Reconfiguration

When demand(P, C) > θ_C, P reconfigures toward P* where:

$P^* = \text{argmin}\{ \text{demand}(P', C) : P' \text{ reachable from } P \text{ via } \Gamma_C \}$

Reconfiguration is guided by two constraints simultaneously: (a) H_P: the loaded history constrains what P can become (a pattern cannot reconfigure against its own history); (b) Γ_C: available gradients constrain what P can move toward (reconfiguration can only use available gradient fields).

Reconfiguration moves toward the nearest coherent state relative to both the loaded history and the available gradients. This is gradient descent over the space of patternings. Section 13.6 (optimisation) is the calculus instance of this.

2.8 Complexity Growth and Fixed Points

Observation 2.1 — Complexity Growth

Any pattern with $L_P > 0$ propagating through any context accumulates loaded history at each

step: $L_{P'} \geq L_P$.

Complexity grows unless P reaches a fixed point of Γ_C: fixed point: $P /_C G = P$ for some $G \in \Gamma_C$ (propagation adds no new history). Fixed points are the stable simple structures of the mechanism: logic: identity (G_{id}), tautologies; calculus: exp (d/dx[exp(x)] = exp(x) — load equals value). Every other pattern is in motion: toward a fixed point, away from one, or in reconfiguration.

2.9 Maximum Complexity and Outward Propagation

Observation 2.2 — Maximum Complexity

Maximum complexity occurs when H_P includes H_P: the pattern's loaded history references itself.

Gradient demand = demand to propagate the full history, including the demand itself. When the gradient field appending the self-referential record weights that record by the current L_P, each propagation step doubles the accumulated demand:

$L_P \rightarrow L_P + L_P \rightarrow \cdots \rightarrow \infty$

No finite coherence threshold contains this demand. Reconfiguration pressure is maximum. The self-referential core cannot cohere within C. Simpler patterns — lower loaded history, faster rate — reconfigure outward from the relational drag of the complex core. The complex centre generates a simple periphery.

This structure appears identically in three settings: in logic (the Gödel sentence, Section 8.4); in calculus (divergent series, Section 13.5); in the structure of logics themselves (pressure toward larger carriers, Section 11). All three are the mechanism at its limit in different carrier settings.

3. Logical Connectives as Gradient Fields

Over a two-element carrier in the zero-drag regime, the available gradient fields take specific forced forms. Because designation is carried by v_P and loaded history by L_P, each gradient field specifies both transformations independently. These gradient fields are not axioms and they are not chosen to match any prior system. They are the coherent forced choices given the carrier arithmetic and the zero-drag regime.

Before giving the definitions, a note on the G/P distinction (Section 2.6): the gradient fields defined below — G_{neg}, G_{and}, G_{or}, G_{imp} — are being treated here as objects of study, patterns in the space of gradient families. They occupy the G role in logical propagation events and simultaneously occupy the P role in this paper's meta-propagation. The naming is post-cut. The gradient fields are pre-cut. And the names themselves are patterns propagating through a linguistic meta-gradient.

3.1 Negation

Definition 3.1 — Negation Gradient

$G_{neg}(v, L) = (1 - v, L)$

Negation flips the designation component and preserves the load. neg P := $P /_C G_{neg} = (1 - v_P, L_P)$.

Double negation: $G_{neg}(G_{neg}(v, L)) = (1-(1-v), L) = (v, L) = P$. The value flip is an involution; loaded history is unchanged. □

3.2 Conjunction

Definition 3.2 — Conjunction Gradient

$G_{and}(P, Q) = (v_P \cdot v_Q, L_P + L_Q)$
Conjunction multiplies designations and adds loads. $P \wedge Q$ is valid iff both $v_P = v_Q = 1$ AND $L_P + L_Q \leq \theta_C$.

3.3 Disjunction

Definition 3.3 — Disjunction Gradient

$G_{or}(P, Q) = (\max(v_P, v_Q), \min(L_P, L_Q))$
Disjunction takes the maximum designation and minimum load: propagates along the path of least gradient demand. $P \vee Q$ is valid iff $\max(vP, v_Q) = 1$ AND $\min(L_P, L_Q) \leq \theta_C$.

3.4 Implication

Definition 3.4 — Implication Gradient

$G_{imp}(P, Q) = Q$ if $v_P = 1$ (P designated: forcing case)
$= (1, 0)$ if $v_P = 0$ (P undesignated: vacuous case)
Implication is activated by designation, not load. $P \rightarrow Q$ is valid when: if $v_P = 1$ then valid(Q); otherwise vacuously valid.

Definition 3.5 — Biconditional

$P \leftrightarrow Q := (P \wedge Q) \vee (\neg P \wedge \neg Q)$. Equivalently: $v_P = v_Q$ AND $|L_P - L_Q| \leq \varepsilon$. P and Q are biconditionally related when they carry the same designation and equivalent load profiles.

Table 1 summarises the gradient field definitions. Value rules determine designation; load rules track propagation cost. Validity requires $v = 1$ and $L \leq \theta_C$.

Connective	Value rule	Load rule	Validity condition
`neg P`	`1 − v_P`	`L_P`	`v_P=0, L_P ≤ θ`
`P ∧ Q`	`v_P · v_Q`	`L_P + L_Q`	`both v=1, sum ≤ θ`
`P ∨ Q`	`max(v_P,v_Q)`	`min(L_P,L_Q)`	`some v=1, min ≤ θ`
`P → Q`	`v_Q if v_P=1, else 1`	`L_Q if v_P=1, else 0`	`v_P=1 ⇒ valid(Q)`
`P ↔ Q`	`1 iff v_P=v_Q`	`|L_P−L_Q|`	`same designation`

Table 1: Gradient field definitions for logical connectives over V = {0,1}.

4. Classical Laws as Forced Boundary Conditions

Propagation over a two-element carrier with a full gradient family produces three boundary conditions. These are what classical logic formalises as its foundational laws — but the formalisation is post-cut. The laws do not pre-exist the propagation mechanism. They emerge as what the carrier arithmetic forces.

4.1 Law of Identity

Theorem 4.1 — Identity

G_{id} is the zero-cost gradient field: $G_{id}(v, L) = (v, L)$. For any pattern P: $P /_C G_{id} = P$.

Proof: G_{id} leaves v_P and L_P unchanged by definition. The encounter with G_{id} adds no loaded history and changes no designation. This is a property of what G_{id} does, not a property of the pattern as substance. □

4.2 Law of Non-Contradiction

Theorem 4.2 — Non-Contradiction (forced by V = {0,1})

For any pattern P with $v_P \in \{0,1\}$: $P \wedge \neg P$ is never designated.

Proof: $G_{and}(P, G_{neg}(P)) = G_{and}((v_P, L_P), (1-v_P, L_P)) = (v_P \cdot (1-v_P), 2 \cdot L_P)$. $v_P \cdot (1-v_P) = 0$ for all $v_P \in \{0,1\}$. This is an arithmetic fact about the two-element carrier. It is not an axiom. It cannot be otherwise in this space. The load $2 \cdot L_P$ also exceeds $\theta_C = 1$ for any active pattern, so $P \wedge \neg P$ is incoherent as well as undesignated. □

4.3 Law of Excluded Middle

Theorem 4.3 — Excluded Middle (forced by V = {0,1})

For any pattern P with $v_P \in \{0,1\}$ and $L_P \leq \theta_C$: $P \vee \neg P$ is always valid.

Proof: $G_{or}(P, G_{neg}(P)) = G_{or}((v_P, L_P), (1-v_P, L_P)) = (\max(v_P, 1-v_P), \min(L_P, L_P)) = (1, L_P)$. $\max(v_P, 1-v_P) = 1$ for all $v_P \in \{0,1\}$. One of the two elements is always 1. The carrier has no gaps. Load: $\min(L_P, L_P) = L_P \leq \theta_C$. Coherent. □

Together, these are the complete boundary conditions of propagation over {0,1} with a full gradient family. What formal systems call classical logic is the exhaustive account of what coherent propagation looks like when parsed through the two-element subject-predicate cut. It is not a choice and not an axiom system — it is what the carrier arithmetic forces, named after the cut has been applied.

5. Quantifiers as Context-Wide Propagation

Quantifiers extend propagation over indexed families of contexts. No new primitives are introduced. The load rules follow directly from conjunction and disjunction: universality requires the hardest instance to cohere; existence requires only the easiest.

Definition 5.1 — Universal Quantification

$\forall x, P(x) := \bigwedge_{x \in D} (P(x) /_{Cx} G_{stable})$
Value: $v_\forall = 1$ iff $v_{P(x)} = 1$ for all $x \in D$.
Load: $L_\forall = \sup_{x \in D} L_{P(x)}$ (gradient demand of the hardest instance).
Valid iff $v_\forall = 1$ and $L_\forall \leq \theta_C$.

Definition 5.2 — Existential Quantification

$\exists x, P(x) := \bigvee_{x \in D} (P(x) /_{Cx} G_{stable})$
Value: $v_\exists = 1$ iff $v_{P(x)} = 1$ for some $x \in D$.
Load: $L_\exists = \inf_{x \in D} L_{P(x)}$ (gradient demand of the easiest instance).
Valid iff $v_\exists = 1$ and $L_\exists \leq \theta_C$.

Quantifier duality follows from the value and load rules together: $\neg(\forall x, P(x))$ holds iff some $P(x)$ is undesignated or its load exceeds the threshold, i.e., $\exists x, \neg P(x)$ is valid. The duality is a propagation observation, not a separate axiom.

6. Inference as Validity-Preserving Propagation

Definition 6.1 — Inference Rule

An inference rule is a gradient field G_{rule} such that: premises $/_C$ G_{rule} $\rightarrow$ conclusion, with valid_C(conclusion) whenever each premise is valid in C. An inference rule is valid in C if it preserves validity.

6.1 Modus Ponens

Theorem 6.1 — Modus Ponens

If $\text{valid}_C(P)$ and $\text{valid}_C(P \rightarrow Q)$, then $\text{valid}_C(Q)$.
Proof: $\text{valid}_C(P)$ means $v_P = 1$ and $L_P \leq \theta_C$. $v_P = 1$, so $G_{imp}(P, Q)$ returns Q (forcing case). $\text{valid}_C(P \rightarrow Q)$ means the implication holds and Q is valid. Therefore $\text{valid}_C(Q)$. Activation is value-based ($v_P = 1$), not load-based. A valid premise is always active. □

Theorem 6.2 — Hypothetical Syllogism

If $\text{valid}_C(P \rightarrow Q)$ and $\text{valid}_C(Q \rightarrow R)$, then $\text{valid}_C(P \rightarrow R)$.
Proof: If $v_P = 1$: $G_{imp}(P, Q) = Q$, and $v_Q = 1$. Then $G_{imp}(Q, R) = R$. Composition: P forces R directly. □

Definition 6.2 — Proof

A proof in context C is a finite sequence of propagation steps:
premises $/_C$ $G_1 \rightarrow P_1$ $/_C$ $G_2 \rightarrow P_2 \cdots /_C$ $G_n \rightarrow$ conclusion
where every intermediate pattern P_i satisfies $\text{valid}_C(P_i)$.

7. Modal Logic as Context Structure

Modal logic emerges as a forced consequence of treating the accessibility relation between possible worlds as a graph of gradient connections between contexts. The load rules for modal operators follow directly from the quantifier load rules: necessity is universality over the accessible context graph, possibility is existence over it.

Definition 7.1 — Necessity

$\Box P := P /_C G_\Box$ where $G_\Box$ propagates P across ALL contexts C′ accessible from C.
Value: $v_{\Box P} = 1$ iff $v_P = 1$ in every accessible C′.
Load: $L_{\Box P} = \sup_{C' \in \Gamma C} L_{P \text{ in } C'}$ (demand of the hardest accessible context).

Definition 7.2 — Possibility

$\Diamond P := P /_C G_\Diamond$ where $G_\Diamond$ checks SOME accessible context C′.
Value: $v_{\Diamond P} = 1$ iff $v_P = 1$ in at least one accessible C′.
Load: $L_{\Diamond P} = \inf_{C' \in \Gamma C} L_{P \text{ in } C'}$ (demand of the easiest accessible context).

Necessity costs more than possibility: sup L ≥ inf L always, so □P carries at least as much gradient demand as ◇P. The Kripke accessibility relation is the graph structure of Γ_C. Modal axiom systems are statements about the topology of this graph:

K: $\Box(P \to Q) \to (\Box P \to \Box Q)$. Forced by any graph.
S4: $\Box P \to \Box\Box P$. Forced by transitive graphs: sup over transitive closure bounded by sup over direct set.
S5: $\Diamond P \to \Box\Diamond P$. Forced by equivalence-class graphs: inf same from every context.

These are not three competing logics. They are three graph topology settings. The boundary conditions come with the topology.

8. Meta-Logical Properties

8.1 Consistency

Theorem 8.1 — Consistency

No context C admits both $\text{valid}_C(P)$ and $\text{valid}_C(\neg P)$ for any pattern P.
Proof: $\text{valid}_C(P)$ requires $v_P = 1$. $\text{valid}_C(\neg P)$ requires $v_{\neg P} = 1 - v_P = 0$. $v_P = 1$ and $1 - v_P = 1$ is impossible. □

8.2 Soundness

Theorem 8.2 — Soundness

Every pattern provable from valid premises is valid in every model context.
Proof: By induction on proof length. Base: premises are valid by hypothesis. Step: each gradient field (Definition 6.1) preserves validity. The value rules of G_{neg}, G_{and}, G_{or}, G_{imp}

reproduce the classical truth functions (verified in Appendix A). □

8.3 Completeness

Theorem 8.3 (Completeness, informal). Every pattern that is valid in all model contexts is provable from the seed gradients by a finite validity-preserving propagation path. This corresponds to the standard Henkin completeness construction: a context in which no further valid patterns can be added without exceeding θ_C is a Henkin context under the PL interpretation. Every pattern valid across all such maximal contexts has a finite propagation path to it.

8.4 Gödel Incompleteness

Gödel incompleteness is not a separate theorem that happens to resemble the mechanism. It is Observation 2.2 — maximum complexity — instantiated in a context that can encode its own propagation history.

Theorem 8.4 — Incompleteness (mechanism statement)

Let C satisfy Definition 8.1 (sufficient expressive power). Then C contains a pattern G such that:

(i) G is valid in the metatheory

(ii) G is not reachable by any finite validity-preserving propagation path within C's own gradient family Γ_C

Construction: The Diagonal Lemma yields G such that $G \equiv \neg\text{Provable}_C(\langle G \rangle)$.

In mechanism terms (Observation 2.2): H_G includes H_G — the history is self-referential. demand(G, C) grows without bound: $L_G \to L_G + L_G \to \cdots$. No finite θ_C contains this demand. Simpler patterns — theorems with bounded loaded history — propagate outward from the boundary of provability. The self-referential core generates the simple periphery. □

Definition 8.1 — Sufficient Expressive Power

C has sufficient expressive power iff C can encode: (a) the natural numbers; (b) the primitive recursive functions; (c) its own proof predicate $\text{Provable}_C(n)$. Satisfied by any context at least as strong as Peano Arithmetic.

The G/P distinction (Section 2.6) is relevant here: the Incompleteness Theorem itself is a pattern propagating through a meta-context — it is P/G→Q running one level above C. Gödel's proof is not a view from outside the mechanism; it is an instance of the mechanism in a meta-context whose gradient family can reference C's own provability predicate. The proof that G is valid in the metatheory is itself a propagation event. This does not undermine the result — it locates it correctly: as a demonstration that Observation 2.2 applies to formal systems, made from within a larger formal system.

8.5 The Value Projection and Formal Logic

Theorem 8.5 — Value Projection

When PL's two-dimensional validity is projected onto the value dimension alone — when the load dimension is discarded and the two-element subject-predicate cut applied — the resulting pattern of valid and invalid designations is precisely what classical propositional logic formalises.

Proof sketch: G_{neg} value rule: 1−v. Projected: the designation pattern formal systems name NOT. G_{and} value rule: $v_P \cdot v_Q$. Projected: AND. G_{or} value rule: $\max(v_P, v_Q)$. Projected: OR. G_{imp} value rule: ($v_P=1 \Rightarrow v_Q$, else 1). Projected: IF. Classical propositional logic is the formal system built on this projection. The load dimension tracks the propagation cost the projection discards. □

9. Probability as Measure over Valid Contexts

Definition 9.1 — Probability

Let $(\Omega, \mathcal{F}, \mu)$ be a measure space of contexts where $\mu(\Omega) = 1$.
Measurability assumption: for every pattern P, the set $\{ C \in \Omega : \text{valid}_C(P) \}$ is $\mathcal{F}$-measurable.
$\Pr(P) := \mu(\{ C \in \Omega : \text{valid}_C(P) \})$
The probability of P is the measure of contexts in which P is valid.

Theorem 9.1 — Kolmogorov Axioms

(1) $\Pr(P) \in [0,1]$ — measure is normalised.
(2) $\Pr(\text{tautology}) = 1$ — valid in all contexts.
(3) $\Pr(P \vee Q) = \Pr(P) + \Pr(Q)$ — when P and Q are mutually exclusive.
Proof: (1) follows from $\mu(\Omega) = 1$ and $\mu \geq 0$. (2) A tautology is valid in every context: $\{C : \text{valid(tautology)}\} = \Omega$ and $\mu(\Omega) = 1$. (3) P and Q mutually exclusive means no context admits both valid(P) and valid(Q). By Theorem 8.1 (Consistency), the valid-context sets are disjoint. Disjointness + μ-additivity gives $\Pr(P \vee Q) = \Pr(P) + \Pr(Q)$. □

Probability in PL is not a degree of belief, nor a frequency. It is the proportion of contexts in which a pattern is valid — how widely coherent propagation is possible. These are not the Kolmogorov axioms imported from probability theory. They are the boundary conditions forced by applying a normalised measure to the space of valid contexts.

10. Paradoxes as Failed Reconfigurations

10.1 The Liar Paradox

The Liar pattern L satisfies $L = \neg L$, requiring $v_L = 1 - v_L$. This equation has no solution in {0,1}: $v = 1 - v$ forces $v = 0.5$, which is outside the Boolean carrier. No propagation of the Liar through any Boolean context reaches a designated output. The failure is entirely in the value component — the designation relation cannot cohere. The load component is unconstrained (L = L trivially).

In terms of the G/P distinction: the Liar is a pattern that tries to occupy both P and G roles simultaneously in the same interaction, using itself as the gradient for its own transformation. This self-application collapses the local cut (Section 2.6): there is no separation between the pattern undergoing transformation and the gradient specifying it. The result is not a paradox requiring resolution — it is a propagation event that cannot cohere in the two-element carrier, identified mechanically.

10.2 Russell's Paradox

The Russell pattern $R = \{x : x \notin x\}$ requires its membership gradient to include its own non-membership gradient. No propagation through any context admitting self-membership reaches a coherent output. The resolution — type theory, ZF — is context stratification: a new context C' separates gradient demands by level. PL makes explicit why stratification works: it separates demands that cannot simultaneously cohere. The Russell pattern is another instance where the G/P local cut collapses: the membership criterion attempts to specify its own gradient at the same level at which it propagates.

10.3 Zeno's Paradoxes

Zeno's paradoxes are infinite propagation chains whose partial loads converge. The series $1/2 + 1/4 + 1/8 + \cdots$ has accumulated load approaching 1, which is within any finite coherence threshold. The paradox disappears: the series is not an obstacle to reaching the endpoint — it is the endpoint's loaded history.

11. Propagation Settings: How Boundary Conditions Set the Gradients

The mechanism has three free parameters: carrier V, gradient family Γ_C, and coherence threshold θ_C. These are inputs — chosen, not derived. What the mechanism then produces is not chosen. It is what the carrier arithmetic and available gradient fields make coherent.

A note on the G/P distinction as it applies to this section: when we say 'choose Γ_C', we are treating gradient families as objects of selection — patterns in a meta-carrier of formal specifications. The choice is itself a propagation event in a meta-context. This is not a regress problem; it is the mechanism operating at multiple levels simultaneously, which is its normal behaviour.

11.1 Two-Element Carrier, Full Gradient Family ($V = \{0,1\}$, $\theta = 1$)

The carrier contains exactly two designation values. The full gradient family means every gradient field definable over this carrier is available. The carrier arithmetic forces two boundary conditions: $v \cdot (1-v) = 0$ (no element equals its own negation — non-contradiction) and $\max(v, 1-v) = 1$ (one of any two complementary elements is always designated — excluded middle). These are arithmetic facts about {0,1}, not axioms.

11.2 Two-Element Carrier, Constructive Gradient Family ($V = \{0,1\}$, $\theta = 1$)

Same carrier. Different gradient family. No direct-assertion gradient. A pattern is valid only if a propagation path reaches it from the seed via the available gradients. $P \vee \neg P$ has value

max(1,0)=1 — the carrier arithmetic is unchanged — but the disjunction gradient requires at least one disjunct to have been reached by propagation. ¬P has no propagation path from the seed in the constructive gradient family. So P∨¬P is not reachable, not because its value is wrong but because there is no propagation path to it. What formal systems have named intuitionistic logic is this setting.

11.3 Extended Threshold (V = {0,1}, θ > 1)

Same carrier. Full gradient family. Threshold raised to θ = 2. The carrier arithmetic is unchanged: v·(1−v) = 0 still holds. P∧¬P is still never designated. What changes is coherence: load = 2.0 ≤ θ = 2.0, so the pattern is coherent-and-undesignated, not incoherent-and-undesignated. What formal systems have named paraconsistent logic is this setting.

11.4 Context Graph Topology: Three Modal Settings

The modal gradient fields $G_\Box$ and $G_\Diamond$ propagate a pattern across the graph of accessible contexts. Three topology settings force three distinct sets of boundary conditions:

Any graph: sup ≥ inf always. □P→◇P forced. (K axiom)

Transitive graph: sup over transitive closure bounded by direct sup. □P→□□P forced. (S4 axiom)

Equivalence-class graph: inf same from all contexts. ◇P→□◇P forced. (S5 axiom)

These are not three competing logics requiring adjudication. They are three graph topology settings. The boundary conditions come with the topology.

11.5 Measure over Valid Contexts: Probability

Probability in the mechanism is what emerges when the space of contexts is given a measure. The Kolmogorov axioms are not imported from probability theory — they are the boundary conditions forced by applying a normalised measure to the space of valid contexts (Section 9).

11.6 Summary: Parameters and Forced Boundary Conditions

V	Γ_C	θ_C	Graph	Boundary conditions forced
{0,1}	Full	1	—	v·(1−v)=0. max(v,1−v)=1. Full assertibility. (Classical logic)
{0,1}	Constr.	1	—	Same arithmetic. Patterns valid only if propagation path from seed. (Intuitionistic)
{0,1}	Full	>1	—	v·(1−v)=0 still. Compound load coherent. Contradiction coherent-undesignated. (Paraconsistent)
{0,1}	G□,G◇	1	Any	sup≥inf. □P→◇P forced. (K)
{0,1}	G□,G◇	1	Trans.	Transitive closure bounded. □P→□□P forced. (S4)
{0,1}	G□,G◇	1	Equiv.	inf same from all. ◇P→□◇P forced. (S5)
{0,1}+μ	Full+μ	1	—	Kolmogorov conditions forced by normalised measure. (Probability)
ℝ	Arith.	ε	—	Product rule, chain rule, FTC forced. (Calculus, Section 13)

Table 2: Propagation settings and the boundary conditions each forces. Parameters are inputs; boundary conditions are what differential propagation produces in that setting.

12. Computational Verification

The following Python implementation verifies the structural claims of PL. All gradient fields, inference rules, and meta-logical properties are confirmed. Patterns use non-negative load throughout. Validity requires both designation (val=1) and coherence (load ≤ threshold).

```
from dataclasses import dataclass
from typing import Any

@dataclass
class Pattern:
    val:  int    # designation component: 0 or 1
    load: float  # gradient demand: always >= 0

@dataclass
class Context:
    threshold: float
    def support(self, p):    return min(p.load, self.threshold)
    def demand(self, p):     return max(0.0, p.load - self.threshold)
    def designated(self, p): return p.val == 1
    def coherent(self, p):   return p.load <= self.threshold
    def valid(self, p):      return self.designated(p) and self.coherent(p)

# Gradient fields — themselves patterns in the meta-carrier
# (Section 2.6: G is P post-boundary-imposition)
def G_id (p):    return Pattern(p.val, p.load)
def G_neg(p):    return Pattern(1 - p.val, p.load)  # flip value, keep load
def G_and(p, q): return Pattern(p.val * q.val, p.load + q.load)
def G_or (p, q): return Pattern(max(p.val, q.val), min(p.load, q.load))
def G_imp(p, q):                  # value-based activation
    if p.val == 1: return q        # designated: force Q
    return Pattern(1, 0.0)         # undesignated: vacuous

def verify_all():
    C = Context(threshold=1.0)
    P = Pattern(1, 1.0)
    Q = Pattern(1, 1.0)

    assert G_id(P) == P                              # identity
    assert G_neg(G_neg(P)) == P                      # double negation
    conj = G_and(P, G_neg(P))
    assert conj.val == 0 and not C.designated(conj)  # non-contradiction
    disj = G_or(P, G_neg(P))
    assert disj.val == 1 and C.valid(disj)           # excluded middle
    assert C.valid(P) and G_imp(P, Q) == Q           # modus ponens
    assert G_imp(Pattern(0, 1.0), Q) == Pattern(1, 0.0)  # vacuous
    assert G_imp(G_imp(P, Q), Q) == Q                # hyp. syllogism
    assert not (C.valid(P) and C.valid(G_neg(P)))    # consistency
    print('All boundary conditions verified.')

verify_all()
```

Listing 1: Complete Propagation Logic implementation. Gradient fields are defined as functions (patterns in the meta-carrier) — consistent with Section 2.6.

13. Calculus as Differential Propagation over a Real Carrier

Calculus is differential propagation (Section 2) with carrier $V = \mathbb{R}$ and continuous gradient fields. Every definition in Section 2 applies without modification: loaded history, gradient demand, propagation rate, relational drag, reconfiguration, complexity growth, fixed points, and maximum complexity. The operator $P/G \to Q$ is unchanged. Only the carrier and gradient family differ.

The G/P distinction (Section 2.6) applies here as it does in logic: the gradient fields G_{pow}, G_{mul}, G_{exp}, G_{sin} are themselves patterns — objects in the space of calculus gradient families, propagating through a meta-context when we define and study them. When we differentiate $f(x) = x^2 \cdot \sin(x)$, we are applying G_{pow} and G_{sin} and G_{mul} to the pattern $(x, 1)$. In another context, G_{pow} itself can be the P undergoing study (as when we ask: what is the fixed point of G_{diff}? — precisely exp). The gradient role and pattern role are contextually assigned throughout.

A note on scope: this section provides a re-interpretation of standard calculus results within the PL framework. It is not an independent derivation of calculus from axioms different from standard analysis. The results are confirmed against analytical values to machine precision (Section 13.4).

13.1 Loaded Patterns in the Calculus Context

Definition 13.1 — Calculus Loaded Pattern

In the calculus regime, a loaded pattern is a pair: $P = (v_P, L_P)$ with $v_P \in \mathbb{R}$, $L_P \geq 0$.

v_P is the current value of the expression. L_P is the differential load: the magnitude of accumulated propagation history. This is Definition 2.1 with $V = \mathbb{R}$. No new primitives are introduced.

13.2 Differentiation as Instantaneous Load Read

Definition 13.2 — Differentiation Gradient Fields

Each arithmetic operation specifies a load-combination rule (the effect of the gradient on L):

```
G_pow(P, n):    v = v_P^n,              L = L_P · n · v_P^(n−1)
G_mul(P, Q):    v = v_P · v_Q,           L = L_P·v_Q + v_P·L_Q
G_div(P, Q):    v = v_P / v_Q,           L = (L_P·v_Q − v_P·L_Q) / v_Q²
G_sin(P):       v = sin(v_P),            L = L_P · cos(v_P)
G_cos(P):       v = cos(v_P),            L = −L_P · sin(v_P)
G_exp(P):       v = exp(v_P),            L = L_P · exp(v_P)
G_log(P):       v = log(v_P),            L = L_P / v_P
```

Seed: input x is initialised as (val=x, load=1). The derivative of f(x) is the load of the output pattern.

Theorem 13.1 — Product Rule

$G_{mul}(P, Q) = (v_P \cdot v_Q, L_P \cdot v_Q + v_P \cdot L_Q)$. This is $d/dx[f \cdot g] = f' \cdot g + f \cdot g'$.

Proof: the load rule for multiplication is the only coherent way to propagate gradient demand through a product. Each factor's load is scaled by the other factor's value. The rule falls out of how multiplication combines two loaded patterns. □

Theorem 13.2 — Chain Rule

For composition f(g(x)): $L_{f\circ g} = L_g \cdot (L_f$ evaluated at g(x)). This is d/dx[f(g(x))] = f′(g(x)) · g′(x).
Proof: the outer gradient demand is scaled by the inner loaded history. The outer pattern propagates through the context established by the inner pattern's output. □

The exp function is the fixed point of G_{exp}: $L_{exp} = \exp(v_P) \cdot L_{input} = v_{exp} \cdot L_{input}$. Load equals value at every step. This is why exp is its own derivative — it is the pattern that propagates without changing the ratio of gradient demand to state. It is the fixed point of the differentiation gradient in the calculus carrier, structurally parallel to identity (G_{id}) in the logic carrier.

13.3 Integration as Load Accumulation to Coherence

Definition 13.3 — Integration Gradient

$\int_a^b f\,dx := f /_C G_{int}$. G_{int} accumulates load across [a,b] using Simpson's rule. Accumulation halts when successive refinements add less than θ_C.

Theorem 13.3 — Fundamental Theorem of Calculus

$d/dx\,[\int_a^x f(t)\,dt] = f(x)$.
Proof: Integration builds accumulated load across [a,x]. Differentiation reads the load added at the endpoint x. The endpoint load is $f(x)\cdot L_{input} = f(x)$ (with unit seed). The accumulation gradient and the read gradient are inverses. □

The FTC is the statement that opposite gradient directions compose to the identity. This is structurally identical to double negation in the logical regime: ¬¬P = P because G_{neg} applied twice returns the original value. d/dx[∫f dx] = f because G_{diff} applied after G_{int} returns the original load. Both are instances of the same property of the propagation operator: opposite gradients cancel.

13.4 Verification of Calculus Gradient Fields

Operation	Expression	Mechanism result	Analytical	Error
Product rule	$d/dx[x^2\sin(x)]$ x=1	2.2232442755	2.2232442755	0.00e+00
Chain rule	$d/dx[\sin(x^2)]$ x=1	1.0806046117	1.0806046117	0.00e+00
Exp fixed pt	d/dx[exp(x)] x=1	val=load=e	val=load=e	0.00e+00
FTC	$d/dx[\int_0^x t^2\,dt]$ x=2	4.0000000000	4.0	0.00e+00
Integration	$\int_0^1 x^2\,dx$	0.3333333333	1/3	0.00e+00
Integration	$\int_0^e \sin(x)\,dx$	2.0000000003	2	2.52e−10
Integration	$\int_0^1 \exp(x)\,dx$	1.7182818290	e−1	5.69e−10
Newton	root of $x^2-2=0$	1.4142135624	√2	0.00e+00

Table 3: Verification of calculus gradient fields. Differentiation errors are zero to machine precision. Integration errors reflect adaptive Simpson convergence.

13.5 Unification Theorem

Theorem 13.4 — Logic-Calculus Unification

Logic and calculus are both instances of differential propagation (Section 2) over carrier sets of different granularity.

Every calculus operation is differential propagation with $V = \mathbb{R}$. Every logical connective is differential propagation with $V = \{0,1\}$. The two subjects differ only in: (i) the carrier set V ($\mathbb{R}$ vs $\{0,1\}$); (ii) the gradient families admitted in Γ_C.

The mechanism — loaded history, gradient demand, propagation rate, relational drag, reconfiguration, complexity growth, fixed points, maximum complexity and outward propagation — is identical in both.

Proof: Sections 3–9 establish the logical case ($V=\{0,1\}$). Sections 13.1–13.4 establish the calculus case ($V=\mathbb{R}$). Both are instances of Definitions 2.1–2.6 with different parameter choices. No additional axioms are required. □

The product rule and the law of non-contradiction are both consequences of how loaded history propagates through binary operations. The Taylor series and Gödel incompleteness are both instances of Observation 2.2 — maximum complexity generating outward propagation of simpler structure. Fixed points (tautologies in logic, exp in calculus) are the stable simple structures at which the propagation rate is maximal and loaded history stops growing. The separation of these subjects into distinct disciplines is an artifact of history, not structure.

13.6 Optimisation as Reconfiguration

Definition 2.6 states that when demand(P, C) exceeds the coherence threshold, P reconfigures toward $P^* = \text{argmin}\{ \text{demand}(P', C) \}$. In the calculus regime ($V = \mathbb{R}$), this is root-finding: a pattern $P = (f(x), f'(x))$ encodes both value and load; coherence requires $v_P = 0$; reconfiguration step moves x toward the nearest coherent state:

Definition 13.4 — Optimisation Reconfiguration Step

Given $P = (f(x), f'(x))$ with $f'(x) \neq 0$:

$x_{n+1} = x_n - v_P / L_P = x_n - f(x_n) / f'(x_n)$

This is Definition 2.6 instantiated in the calculus regime. No new primitive is introduced.

Theorem 13.5 — Quadratic Convergence of Reconfiguration

Let f be twice differentiable near its root x^* with $f'(x^*) \neq 0$. Then:

$\text{demand}(P_{n+1}, C) \approx (f''(x^*) / 2f'(x^*)) \cdot \text{demand}(P_n, C)^2$

Each reconfiguration step squares the demand. Four steps reduce demand from 10^{-1} to below 10^{-12}. □

14. Conclusion

Propagation Logic is built on a single dynamical mechanism: relational patternings propagate through contexts accumulating loaded history, which sets their gradient demands for further

propagation. Patterns with less loaded history propagate fastest. Complexity builds through propagation and generates relational drag. When demands exceed available gradients, patterns reconfigure toward coherence. At maximum complexity — when a pattern's history includes itself — reconfiguration pressure becomes unbounded, and simpler structures propagate outward.

Logic and calculus are two carrier-set settings of this single mechanism. The boundary conditions that propagation over a two-element carrier with a full gradient family forces — what classical logic formalises after the subject-predicate cut — are not axioms: $v \cdot (1-v) = 0$ and $\max(v, 1-v) = 1$ are arithmetic facts about {0,1}. The rules of calculus are forced by the same mechanism over a real carrier. Every other logic is a different carrier or gradient family setting.

Gödel incompleteness, the divergence of series, and the pressure from classical logic toward richer systems are all Observation 2.2 — the mechanism at its limit — in different carrier settings. Fixed points (tautologies in logic, exp in calculus) are the stable simple structures from which complexity propagates outward, and to which reconfiguration returns.

The G/P Distinction: Closing the Ontological Loop

Version 13 of this paper makes explicit a distinction that earlier versions left implicit and that formal treatments of PL routinely obscure.

Gradient fields G are not a different kind of thing from patterns P. G is a pattern that occupies the gradient role in a specific interaction — a local and momentary contextual assignment, not an intrinsic substance difference. All G is P post-boundary-imposition. When we define G_{neg} or G_{and}, we are treating these as objects of study: patterns in the space of gradient families, propagating through a meta-level context. The gradient role is assigned at the moment of interaction and can be reassigned in the next.

All claims to describe a gradient field are meta-G: any statement about G is itself a propagation event in a meta-context. 'G_{neg} flips the value component' is a pattern propagating through the meta-gradient of formal specification. The description of the mechanism is an instance of the mechanism. There is no view from outside.

The P/G distinction is local and momentary: after an interaction $P/G \rightarrow Q$, the roles can flip. What was G becomes P for the next propagation event. The distinction is a relational cut at the moment of interaction, not a permanent ontological divide.

This matters because without it, gradient fields are smuggled in as hidden substances — a subtle reification that contradicts the paper's founding claim that the mechanism contains no primitive substances. With it, the mechanism is self-consistent: patterns and gradients are the same kind of thing, contextually assigned to different roles in each propagation event. The subject-predicate cut that formal systems apply to patterns can equally be applied to gradients. Both are reifications of aspects of the same propagation process.

The mechanism IS the medium of its own description.

The mechanism in one statement

Relational patternings propagate differentially through contexts, accumulating loaded history that sets their gradient demands, generating relational drag, reconfiguring toward coherence, and producing outward propagation of simpler structure when complexity becomes unstable. Logic, calculus, and the dynamics of complex systems are all special

cases. Choose the carrier and the available gradients. The rest follows. The gradients themselves are patterns — choose them too, and the meta-context generates the object-level context.

References

[1] G. Boolos, J. Burgess, and R. Jeffrey, Computability and Logic, 5th ed. Cambridge University Press, 2007.

[2] S. Kripke, ‘Semantical Considerations on Modal Logic,’ Acta Philosophica Fennica, vol. 16, pp. 83–94, 1963.

[3] K. Gödel, ‘Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I,’ Monatshefte für Mathematik und Physik, vol. 38, pp. 173–198, 1931.

[4] G. Priest, An Introduction to Non-Classical Logic, 2nd ed. Cambridge University Press, 2008.

[5] A. Tarski, ‘The Concept of Truth in Formalized Languages,’ in Logic, Semantics, Metamathematics, pp. 152–278. Oxford University Press, 1956.

[6] D. van Dalen, Logic and Structure, 5th ed. Springer, 2013.

[7] P. Hájek, Metamathematics of Fuzzy Logic. Springer, 1998.

[8] J. Barwise and J. Etchemendy, Language, Proof and Logic. CSLI Publications, 1999.

[9] A. Griewank and A. Walther, Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation, 2nd ed. SIAM, 2008.

[10] W. Rudin, Principles of Mathematical Analysis, 3rd ed. McGraw-Hill, 1976.

[11] J. Pugmire, Mathesis Universalis: Calculus and Number Systems from Differential Propagation. Independent Researcher.

Appendix A. The Value Projection: Designation Patterns over the Two-Element Carrier

The following tables show the value-dimension output of each gradient field when evaluated over V = {0,1} with load dimension discarded. Classical propositional logic is the formal system constructed by treating these patterns as primitive truth functions. The tables do not show PL matching classical logic — they show what classical logic formalises, which is the value projection of PL gradient fields over this carrier.

v_P	v_neg_P = 1−v_P	Classical ¬
0	1	1 ✓
1	0	0 ✓

v_P	v_Q	v_P·v_Q	Classical ∧
0	0	0	0 ✓
0	1	0	0 ✓
1	0	0	0 ✓

1	1	1	1 ✓

v_P	v_Q	v_imp	Active?	Classical →
0	0	1	No (vacuous)	1 ✓
0	1	1	No (vacuous)	1 ✓
1	0	0	Yes ⇒ v_Q=0	0 ✓
1	1	1	Yes ⇒ v_Q=1	1 ✓

Appendix A tables: the designation patterns of the value projection over V={0,1}. Classical propositional logic is the formal system built on these patterns treated as primitive.

Appendix B. Questions That Presume What They Ask About

A question sets boundary conditions on its own answer. When a question's intelligibility requires the truth of a contested premise, any answer stable within that question's frame will confirm the premise — not because the premise was examined, but because it was built into the gradient structure of the question itself.

The following examples are structural demonstrations of the mechanism: each question instantiates a gradient family G that makes discrete substance, intrinsic identity, or pre-cut structure coherent as an answer-form, and incoherent as a target of scrutiny. Each question is itself a propagation event — a P/G→Q event where Q is the form of acceptable answers. The G/P distinction (Section 2.6) applies directly: describing a question's gradient structure is meta-G, itself a propagation event one level up.

B.1 'Relations between what?'

The question presupposes that relata — discrete entities that stand in relations — are prior to and independent of the relations themselves. Within the gradient family the question instantiates, any answer must supply discrete relata or be rejected as incomplete. The question cannot be answered by pointing to relational structure alone. 'What' is a language demand, not an observational one. It is never observed outside the cut.

B.2 'What grounds the continuity of a pattern over time?'

This question reifies time as a container and patterns as substances persisting through it. Within PL, temporal ordering is internal to the propagation sequence: P/G→Q is the reconfiguration relation, and 'before' and 'after' are structural features of that relation, not coordinates in an external medium.

B.3 'What selects the gradient family?'

The question presupposes that Γ_C is a discrete object chosen by some prior agent separate from the propagation it characterises. Within PL, the gradient family is not selected from outside: it is where the cut is made. V, Γ_C, and θ_C are degrees of freedom in how relational structure is parsed. The question's gradient family presupposes the answer it is asking for. The selector is itself a pattern in a meta-context — G/P again.

B.4 'What is consciousness made of?'

The question instantiates a composition gradient family: it treats 'made of' as the appropriate relation and presupposes that the answer will be a set of discrete constituent substances. Any answer that does not supply discrete constituents is rejected by the question's gradient as not answering the question. This is not because those answers are wrong. It is because the question's loaded history makes compositional substance the only stable answer-form.

B.5 'What carries the information?'

The question presupposes a bearer-property structure: a discrete substance that possesses information as an intrinsic property. Within PL, information is not a property carried by a substance — it is loaded history: the accumulated propagation record H_P that constitutes the pattern relationally. There is no carrier separate from the pattern, and no information separate from the propagation.

B.6 The General Structure

Each example has the same structure. A question Q instantiates a gradient family G_Q. G_Q makes a specific answer-form coherent and all other answer-forms incoherent. Any answer that propagates stably through G_Q will confirm the premise embedded in the question. Recognising this does not dissolve the questions. It locates them correctly: as propagation events within specific gradient families, whose answers are stable relative to those families and not beyond them. Every question is a $P/G \rightarrow Q$ event. The question's gradient field is itself a pattern — Section 2.6 applies throughout.

End of paper.